

Modulo 5 — Comunicazione con API e gestione dati

REST • fetch vs axios • async/await • Promise.all • CRUD • error handling •
interceptors • AsyncStorage

Obiettivi del modulo

Cosa saprai fare

- Capire API REST e leggere una documentazione.
- Fare chiamate GET/POST/PUT/DELETE in modo robusto (fetch e axios).
- Gestire asincrono con async/await, Promise.all e varianti.
- Gestire errori, timeout, retry, cancel e stati UI (loading/error).
- Centralizzare configurazione e auth con axios instance + interceptors.
- Gestire dati localmente: cache e preferenze con AsyncStorage.
- Costruire una mini-app completa: lista, dettaglio, create, update, delete.

API REST

Cos'è una risorsa

- Una risorsa è 'qualcosa' che il server gestisce: post, utente, prodotto.
- Di solito ogni risorsa ha un identificatore (id).
- Endpoint esempi: /posts (collezione), /posts/42 (elemento).

HTTP methods

Cosa significano davvero

- GET: leggere dati (non dovrebbe modificare server).
- POST: creare (nuova risorsa).
- PUT: sostituire una risorsa (di solito body completo).
- PATCH: aggiornare parzialmente (solo campi cambiati).
- DELETE: rimuovere una risorsa.

Status code

Regola pratica

- 200 OK: risposta con body.
- 201 Created: creato (spesso con body).
- 204 No Content: ok ma senza body (spesso su delete).
- 400/422: input non valido.
- 401/403: auth/permessi.
- 404: non trovato.
- 500: errore server.

Headers

Cosa sono

- Content-Type: tipo di body inviato (application/json).
- Accept: tipo di risposta attesa.
- Authorization: token/sessione.
- Cache-Control / ETag: caching (avanzato).

JSON

Perché lo vedi ovunque

- È un formato testuale per scambiare dati (oggetti e array).
- In JS lo trasformi con `JSON.stringify` e `JSON.parse`.
- Le API REST spesso rispondono in JSON.

Esempio JSON

Oggetto e array

JSON

```
const post = {  
  id: 1,  
  title: "Hello",  
  tags: ["react", "api"],  
};  
  
const json = JSON.stringify(post);  
const back = JSON.parse(json);
```

- Stringify: JS → stringa. Parse: stringa → JS.

package.json

Perché è importante

- Descrive il progetto (nome, versione) e le dipendenze.
- Contiene gli script: `npm run start` / `dev` / `test`.
- Definisce anche: `type module`, `engines`, ecc. (a seconda del progetto).

fetch()

Come ragionare

- fetch restituisce una Promise che risolve con Response.
- Una risposta 404/500 NON genera eccezione automaticamente.
- Devi: controllare res.ok e poi fare parsing (res.json / res.text).

fetch GET completo

Con res.ok + parsing

fetchJson()

```
async function fetchJson(url) {  
  const res = await fetch(url);  
  
  if (!res.ok) {  
    const text = await res.text();  
    throw new Error(`HTTP ${res.status}: ${text}`);  
  }  
  
  return res.json();  
}
```

fetch POST JSON

Body + headers

POST con fetch

```
async function postJson(url, body) {  
  const res = await fetch(url, {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify(body),  
  });  
  
  if (!res.ok) throw new Error(`HTTP ${res.status}`);  
  return res.json();  
}
```

fetch timeout

Perché serve

- Su rete mobile una request può restare appesa.
- fetch non ha timeout built-in: usi AbortController.

AbortController

Timeout con fetch

Timeout

```
async function fetchWithTimeout(url, ms = 8000) {  
  const ctrl = new AbortController();  
  const id = setTimeout(() => ctrl.abort(), ms);  
  
  try {  
    const res = await fetch(url, { signal: ctrl.signal });  
    return res;  
  } finally {  
    clearTimeout(id);  
  }  
}
```

axios

Perché spesso è più comodo

- Ritorna res.data già pronto (meno boilerplate).
- I non-2xx finiscono nel catch.
- Timeout integrato.
- Interceptors per token, log, refresh, mapping errori.

Install axios

NPM

Install

```
npm install axios
```

api.ts

Instance centrale

api.ts

```
import axios from "axios";

export const api = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com",
  timeout: 8000,
  headers: { "Content-Type": "application/json" },
});
```

axios instance

Cosa ci mettiamo dentro

- `baseUrl`: l'host dell'API.
- `timeout`: limite massimo per la request.
- `headers` di default: `Content-Type`, `Accept`.
- `validateStatus`: regole custom su status (avanzato).

Interceptors

Perché sono utili

- Request interceptor: aggiungi token, locale, correlation id.
- Response interceptor: gestisci 401, normalizzi errori.
- Risultato: le schermate hanno meno codice ripetuto.

Request interceptor

Token

Request interceptor

```
api.interceptors.request.use((config) => {  
  const token = "TOKEN_DA_STORAGE";  
  config.headers.Authorization = `Bearer ${token}`;  
  return config;  
});
```

Response interceptor

Mapping errori

Response interceptor

```
api.interceptors.response.use(  
  (res) => res,  
  (err) => {  
    const status = err?.response?.status;  
  
    const message =  
      status === 401 ? "Non autorizzato" :  
      status === 404 ? "Risorsa non trovata" :  
      "Errore di rete o server";  
  
    err.userMessage = message;  
    return Promise.reject(err);  
  }  
);
```

Refresh token (idea)

Scenario tipico

- 1) request fallisce con 401
- 2) tenti refresh token (una volta)
- 3) ripeti la request originale
- 4) se fallisce ancora: logout

Asincrono in JS

Concetti base

- Una Promise rappresenta un lavoro che finirà in futuro.
- `await` aspetta il risultato senza bloccare la UI thread.
- `try/catch` gestisce errori asincroni con `await`.

async/await

Esempio

async/await

```
async function load() {  
  try {  
    const res = await api.get("/posts/1");  
    return res.data;  
  } catch (e) {  
    throw e;  
  }  
}
```

Promise.all

Parallelo

- Esegue più promise in parallelo.
- Se una fallisce, fallisce tutto.
- Utile quando le chiamate sono indipendenti.

Promise.all

Posts + users

Promise.all

```
const [postsRes, usersRes] = await Promise.all([
  api.get("/posts"),
  api.get("/users"),
]);

setPosts(postsRes.data);
setUsers(usersRes.data);
```

Promise.allSettled

Quando preferirla

- Vuoi risultati anche se una richiesta fallisce.
- Ogni elemento ha status: fulfilled/rejected.
- Utile per schermate con blocchi indipendenti.

Promise.allSettled

Esempio

`allSettled`

```
const results = await Promise.allSettled([
  api.get("/posts"),
  api.get("/users"),
  api.get("/comments"),
]);

for (const r of results) {
  if (r.status === "fulfilled") console.log(r.value.data);
  else console.log("Errore:", r.reason);
}
```

Error handling

Categorie di errori

- Network error: niente connessione, DNS, timeout.
- HTTP error: 4xx/5xx.
- Parse error: risposta non JSON quando lo aspetti.
- App error: bug nel mapping/trasformazione dati.

Mapping errori (UI)

Da axios error a testo

Error mapping

```
function getErrorMessage(e) {  
  const status = e?.response?.status;  
  if (status === 401) return "Sessione scaduta";  
  if (status === 404) return "Elemento non trovato";  
  if (e?.code === "ECONNABORTED") return "Timeout di rete";  
  return "Errore di rete o server";  
}
```

Cancel request

Perché serve

- Search: annulli la request precedente mentre l'utente digita.
- Navigazione: eviti setState dopo unmount.

Axios + AbortController

Cancel

Cancel

```
let ctrl;  
  
async function search(q) {  
  ctrl?.abort();  
  ctrl = new AbortController();  
  
  const res = await api.get("/posts", { signal: ctrl.signal });  
  return res.data;  
}
```

Liste grandi

Paginazione

- Non vuoi caricare migliaia di elementi insieme.
- Soluzione: paginazione o infinite scroll.
- FlatList: onEndReached per caricare pagine.

Paginazione (idea)

Query params

Pagination

```
const res = await api.get("/items", {  
  params: { page: 1, pageSize: 20 },  
});
```

Debounce search

Concetto

- Riduce le request: aspetta che l'utente smetta di digitare.
- Migliora UX e riduce carico server.

TypeScript e API

Perché aiuta

- Evita errori di proprietà sbagliate.
- Autocompletamento e refactor sicuro.
- Tipi per response e mapping.

Tipo Post

Esempio

Post

```
export type Post = {  
  id: number;  
  userId: number;  
  title: string;  
  body: string;  
};
```

API typed

axios generic

Typed

```
import type { Post } from "./types";

export async function getPosts() {
  const res = await api.get<Post[]>("/posts");
  return res.data;
}
```

CRUD: mini-app

Cosa costruiremo

- Lista post (GET).
- Dettaglio (GET by id).
- Crea (POST).
- Modifica (PUT).
- Elimina (DELETE).
- Cache leggera (AsyncStorage).

API di test

Per esercizi

- JSONPlaceholder: jsonplaceholder.typicode.com
- DummyJSON: dummyjson.com
- Obiettivo: imparare request/response e stati UI.

Endpoint posts

JSONPlaceholder

- GET /posts
- GET /posts/1
- POST /posts
- PUT /posts/1
- DELETE /posts/1

Service: listPosts

postsService.ts

Service

```
export async function listPosts() {  
  const res = await api.get("/posts");  
  return res.data;  
}
```

Service: getPost

postsService.ts

Service

```
export async function getPost(id) {  
  const res = await api.get(`/posts/${id}`);  
  return res.data;  
}
```

Service: createPost

postsService.ts

Service

```
export async function createPost(input) {  
  const res = await api.post("/posts", input);  
  return res.data;  
}
```

Service: updatePost

postsService.ts

Service

```
export async function updatePost(id, input) {  
  const res = await api.put(`/posts/${id}`, { id, ...input });  
  return res.data;  
}
```

Service: deletePost

postsService.ts

Service

```
export async function deletePost(id) {  
  await api.delete(`/posts/${id}`);  
}
```

Separare responsabilità

Pattern

- UI: componenti e presentazione.
- Service: chiamate API.
- Hook: loading/error + esecuzione (riuso).

Hook: useRequest

Schema

useRequest

```
import { useState, useCallback } from "react";

export function useRequest(fn) {
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const run = useCallback(async (...args) => {
    try {
      setLoading(true);
      setError(null);
      return await fn(...args);
    } catch (e) {
      setError(e);
      throw e;
    } finally {
      setLoading(false);
    }
  }, [fn]);

  return { run, loading, error };
}
```

UI: lista con hook

Loading/error

UI pattern

```
const { run, loading, error } = useRequest(listPosts);

useEffect(() => {
  run().then(setPosts).catch(() => {});
}, [run]);

if (loading) return <ActivityIndicator />;
if (error) return <Text>{getErrorMessage(error)}</Text>;
```

Debug richieste

Cosa guardare

- URL completo + params.
- Headers (Authorization, Content-Type).
- Body inviato.
- Status code + response.

Logging interceptor

Request/response

Logging

```
api.interceptors.request.use((config) => {  
  console.log("[REQ]", config.method, config.url, config.params);  
  return config;  
});  
  
api.interceptors.response.use((res) => {  
  console.log("[RES]", res.status, res.config.url);  
  return res;  
});
```

AsyncStorage

Quando usarlo

- Cache per avvio veloce.
- Preferenze (tema, lingua).
- Non è un database: evita dati enormi.

Install AsyncStorage

Expo

Install

```
npx expo install @react-native-async-storage/async-storage
```

Cache helpers

Read/write

Cache

```
import AsyncStorage from "@react-native-async-storage/async-storage";

export async function cacheSet(key, value) {
  await AsyncStorage.setItem(key, JSON.stringify(value));
}

export async function cacheGet(key) {
  const raw = await AsyncStorage.getItem(key);
  return raw ? JSON.parse(raw) : null;
}
```

Cache strategy

Stale-while-revalidate

- 1) leggi cache e mostri subito
- 2) fai fetch dal network
- 3) aggiorni UI e riscrivi cache

SWR (semplice)

Schema

SWR

```
const cached = await cacheGet("posts");  
if (cached) setPosts(cached);  
  
const fresh = await listPosts();  
setPosts(fresh);  
await cacheSet("posts", fresh);
```

Pausa

15 minuti

Riprendiamo tra 15 minuti

Percorso pratico

Passi

- 1) api.ts (axios instance).
- 2) postsService.ts (CRUD).
- 3) Lista con loading/error.
- 4) Dettaglio.
- 5) Create/Update/Delete.
- 6) Cache con AsyncStorage.

Sicurezza (base)

Cosa evitare

- Non hardcodare token/chiavi nel client.
- Server deve verificare permessi.
- UI: messaggi semplici, log tecnici separati.

Riepilogo

Punti chiave

- fetch: standard ma più manuale.
- axios: più comodo + interceptors + timeout.
- Promise.all / allSettled: parallelo e resilienza.
- UI robusta: loading/error/cancel/retry.
- AsyncStorage: cache/preferenze (non DB).

Domande di ripasso

Domanda 1 di 5

1. Quando è meglio usare axios invece di fetch?

- ☐ Quando vuoi gestire errori/timeout/interceptors in modo più semplice
- ☐ Quando vuoi evitare JSON.stringify sempre
- ☐ Quando l'app è solo web
- ☐ Quando non ti serve baseUrl

Domande di ripasso

Domanda 2 di 5

2. Con Promise.all cosa succede se una richiesta fallisce?

- ☐ Le altre continuano e ottieni risultati parziali automaticamente
- ☐ Promise.all fallisce e vai nel catch
- ☐ La UI si blocca per sempre
- ☐ Axios riprova da solo sempre

Domande di ripasso

Domanda 3 di 5

3. Quale pattern mostra subito cache e poi aggiorna dal network?

- ☐ Stale-while-revalidate
- ☐ Hard refresh only
- ☐ No cache
- ☐ Server-side rendering

Domande di ripasso

Domanda 4 di 5

4. Perché serve cancellare una request in una search?

- ☐ Per rendere l'app più lenta
- ☐ Per evitare risultati vecchi e setState dopo unmount
- ☐ Per aumentare i 500 error
- ☐ Per sostituire useEffect

Domande di ripasso

Domanda 5 di 5

5. Cosa fa un axios interceptor di request?

- ☐ Aggiorna automaticamente la UI
- ☐ Aggiunge/modifica config (es. Authorization)
- ☐ Sostituisce AsyncStorage
- ☐ Rimuove tutte le Promise